F25-147-D-Kairo

Project Team

Hafsa Imtiaz 22I-0959
Areen Zainab 22I-1115
Areeba Riaz 22I-1244

Session 2022-2026

Supervised by

Dr. Ali Zeeshan Ijaz

Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

June, 2026

Contents

1 Introduction **1**

1.1 Problem Statement 2

1.2 Scope 2

1.3 Modules 2

1.3.1 Module 1: Meeting Bot Integration 3

1.3.2 Module 2: Transcription and Summarization 3

1.3.3 Module 3: Meeting Memory Engine 3

1.3.4 Module 4: Task Board and Action Item Detection 4

1.3.5 Module 5: Privacy, Control, and Analytics 4

1.4 User Classes and Characteristics 5

1.5 Gantt Chart 7

2 Project Requirements **9**

2.1 Use-case/Event Response Table/Storyboarding 9

2.2 Functional Requirements 14

2.2.1 Module 1: Meeting Bot Integration & Workspace Management . . 14

2.2.2 Module 2: Transcription and Summarization 14

2.2.3 Module 3: Meeting Memory Engine & Knowledge Graph 15

2.2.4 Module 4: Task Board & Action Item Management 15

2.2.5 Module 5: Privacy, Control & Analytics 16

2.3 Non-Functional Requirements 17

2.3.1 Reliability 17

2.3.2 Usability 17

2.3.3 Performance 17

2.3.4 Security 18

2.4 Domain Model 18

3 System Overview **21**

3.1 Architectural Design 21

3.1.1 Presentation Layer 21

3.1.2 Application Layer 22

3.1.3 Data Layer 23

3.2 Design Models 25

3.2.1 Class Diagram 25
3.2.2 Activity Diagram 25
3.2.3 Class-Level Sequence Diagram 27
3.2.4 State-Transition Diagram 31
3.3 Data Design 33
3.3.1 Data Storage Structure 33
3.3.2 Major Entities 33
3.3.3 Data Flow and Processing 33
3.3.4 Summary 35

References **38**

List of Figures

1.1 Gantt Chart 7

2.1 Use-Case Diagram 13
2.2 Domain Model Diagram of the System 20

3.1 System Architectural Design Diagram 24
3.2 Class diagram 25
3.3 Activity Diagram of Kairo showing the meeting lifecycle. 26
3.4 Class-level Sequence Diagram for the registration module, highlighting the interactions between user, authentication service, and workspace components. 27
3.5 Class-level Sequence Diagram illustrating both automatic and manual bot joining processes, followed by transcription initiation. 27
3.6 Class-level Sequence Diagram for visualizing the Knowledge Graph. . . . 28
3.7 Class-Level Sequence Diagram of Live Transcription 28
3.8 Class-level Sequence Diagram illustrating action item detection during meetings. 29
3.9 Class-level Sequence Diagram for meeting summary generation. 29
3.10 Class-level Sequence Diagram depicting Kanban generation within Kairo's task management module. 30
3.11 Class-level Sequence Diagram depicting the retrieval process from Kairo's Meeting Memory using conversational queries and semantic vector search. 30
3.12 Class-level Sequence Diagram illustrating the activation and handling of Privacy Mode in Kairo. 31
3.13 State-Transition Diagram of Kairo showing the end-to-end meeting lifecycle. 32
3.14 Entity-Relationship Diagram 35

List of Tables

1.1 User Classes and Characteristics for Kairo System 6

2.1 Use Cases for Kairo 10
2.1 Use Cases for Kairo 11
2.1 Use Cases for Kairo 12
2.1 Use Cases for Kairo 13

3.1 Key Entities and Their Relationships 34

Chapter 1

Introduction

This document presents the work completed during Iteration #1 of our journey toward developing Kairo, an adaptive meeting intelligence platform designed for corporate teams. This iteration represents the project's foundation, where we defined the problem space, outlined key objectives, and built the initial components that will support Kairo's future capabilities.

The motivation behind Kairo stems from the persistent challenges organizations face in managing meetings, capturing decisions, tracking action items, and ensuring accountability. While tools like Zoom and Google Meet focus on hosting and recording, they often fail to deliver structured outcomes or actionable insights. Prior research highlights that action-item-driven summarization and context modeling remain open problems in meeting analysis and productivity tools [8, 10].

Kairo addresses this gap by integrating real-time transcription, action item detection, task extraction, and Kanban-style task boards, powered by a Meeting Memory Engine that turns conversations into a searchable knowledge base. Recent advances in meeting summarization and AI-assisted collaboration, such as MeetMap and AreWe On Track, demonstrate the growing relevance of LLM-powered meeting comprehension and goal reflection [4, 5].

As remote and hybrid collaboration becomes the norm, the absence of structured meeting memory often leads to inefficiency and repeated discussions. In this first iteration, Kairo focuses on establishing the core workspace and enabling the meeting bot to automatically join scheduled sessions, laying the groundwork for future integration of transcription, summarization, and knowledge graph–based memory systems [6].

1.1 Problem Statement

Meetings are central to teamwork, yet organizations often struggle to turn them into clear and actionable outcomes. Decisions, tasks, and responsibilities are frequently undocumented or scattered across emails, chat logs, or lengthy recordings, making them difficult to retrieve when needed. This lack of continuity leads to repeated conversations, wasted time, and delays in project progress. Existing tools, such as transcription services or note-taking apps, offer only partial solutions, as they capture information but fail to ensure accountability, structured follow-up, or proper task management.

Existing work such as MeetMap [5] has shown promise in using LLMs for real-time collaborative dialogue mapping, indicating that semantic awareness during meetings can substantially improve team engagement and clarity. However, such systems often remain research prototypes and do not address long-term memory or task tracking — key challenges that Kairo aims to solve.

1.2 Scope

The scope of Kairo is to transform corporate meetings into structured, trackable, and actionable outcomes. The system captures conversations, generates accurate transcriptions, and ensures continuity through a **meeting memory engine** that recalls past discussions and links unresolved items. It extracts action items, identifies deadlines, and organizes them on a Kanban-style task board. Post-meeting summaries, calendar-based reminders, and analytics dashboards further enhance accountability and productivity.

Kairo will integrate with platforms like Zoom and Google Meet as a bot participant and provide users with a web and desktop-based dashboard for tasks, summaries, and reminders. The system focuses strictly on meeting intelligence and continuity, excluding unrelated project management features such as HR or financial tracking. To support its memory graph and semantic retrieval functions, Kairo leverages FAISS for vector similarity search and Neo4j for graph-based contextual retrieval [6]. PostgreSQL serves as the primary data store due to its proven scalability and compatibility with AI-driven workloads [7, 13].

1.3 Modules

The system is divided into modules to ensure a modular and structured implementation. Each module focuses on specific functionalities of the system.

1.3.1 Module 1: Meeting Bot Integration

This module allows Kairo to join third-party meeting platforms such as Zoom or Google Meet as a participant. It records audio in real time and prepares the input for transcription.

1. Auto-Join & Capture: Automatically joins scheduled meetings (via calendar integrations), records audio, and starts real-time transcription.
2. Calendar Integrations: Sync with Google/Outlook calendars to auto-detect meetings.
3. Third-Party Tool Integrations: Connect to Google Drive, etc., for seamless push/pull of meeting notes and tasks.
4. Manage session-level authentication and permissions.

This functionality is conceptually aligned with Recall.ai's approach to meeting bot automation [11], enabling autonomous participation across digital meeting platforms.

1.3.2 Module 2: Transcription and Summarization

This module is responsible for converting audio into accurate text and generating concise summaries.

1. Real-Time Transcription (WhisperX-based): Produces accurate transcripts with speaker diarization using Pyannote and Resemblyzer.
2. Summarization Engine: Generates layered summaries (high-level for executives, detailed for participants).
3. Whisper Mode: Provides real-time micro-recaps every 5–10 minutes, displayed in chat/UI so late joiners instantly catch up.
4. Highlight unresolved items from previous meetings using the memory engine.

The transcription layer is built upon WhisperX [1] for high-fidelity speech recognition, while Diart [9] and Resemblyzer [12] provide real-time diarization and speaker embedding support.

1.3.3 Module 3: Meeting Memory Engine

This is the core module of the system that maintains continuity across meetings by storing past discussions and surfacing relevant items during future sessions.

1. Maintain a structured memory of past meeting outcomes.
2. Meeting Memory Graph: Visualize concepts, tasks, and decisions across sessions in an interactive Neo4j-style graph.
3. Link unresolved tasks, emphasize pending discussions, and clearly outline dependencies between action items, discussions, and meetings.
4. Provide contextual continuity across sessions.

The memory engine leverages vector embeddings and graph relationships inspired by recent advancements in conversational memory and retrieval frameworks [2, 6].

1.3.4 Module 4: Task Board and Action Item Detection

This module organizes meeting outcomes into a collaborative interface and identifies actionable responsibilities.

1. Action Item Detection: Automatically extracts tasks, commitments, and deadlines from conversations.
2. Kanban Board Integration: Action items flow directly into Kanban-style boards inside Kairo.
3. Task Contextual Micro-Channels: Each task has a "Meeting Context" tab, collecting discussions, snippets, and decisions tied to it across multiple meetings.
4. Allow updates and edits by team members.

Kairo's action item pipeline is informed by action item detection models discussed in [10] and extended with contextual modeling for better precision.

1.3.5 Module 5: Privacy, Control, and Analytics

This module ensures accountability, compliance, and performance insights.

1. Privacy & Compliance Mode: Toggle transcription/recording off for sensitive parts while still logging metadata, managed by role-based permissions.
2. Role-Based Access Control: Restricts visibility of sensitive meeting data, with different levels of detail for executives, managers, and contributors.
3. Smart Search & Speaker Recall: Retrieve past meeting content by keyword, task, decision, or speaker (e.g., "What did Sarah say about X?") using semantic search.

4. Multimodal Meeting Capture: Captures slides, screenshares, and speech, syncing transcripts with visuals and extracting KPIs, tables, and diagrams into searchable notes.
5. Send follow-up reminders through calendar integration.
6. Provide analytics dashboards for productivity and meeting efficiency.

1.4 User Classes and Characteristics

Team Lead / Manager	A Team Lead or Manager is responsible for creating and managing workspaces within Kairo. They oversee meeting participation, assign roles, and control data permissions. Managers can connect calendars, initiate meetings, trigger manual bot joins, review transcriptions, and approve post-meeting summaries or task extractions. They also manage Kanban boards and track action items across projects. Managers are typically mid-to-senior-level professionals (10–15% of total users). They are frequent users who rely on Kairo for real-time meeting intelligence and workflow coordination.
Employee	Employees are the primary participants in meetings. They interact with Kairo to view meeting summaries, track assigned tasks, contribute to discussions, and access project-related insights. Employees can search past transcripts, update task statuses, and review contextual micro-channels for task-related discussions. They represent the largest user group (approximately 70–75% of total users). They typically access Kairo through their workspace dashboards and integrated meeting tools (e.g., Google Meet, Zoom).
Project Manager	Project Managers use Kairo to monitor progress across multiple meetings, projects, and task boards. They oversee project milestones, assign or update tasks, and analyze performance metrics. They heavily rely on Kairo's summarization, action extraction, and Kanban visualization features to ensure accountability and efficiency. This group (about 10% of users) acts as a bridge between management and employees.
Administrator	Administrators are responsible for overall system configuration, workspace setup, and user access control. They monitor integrations (Google Calendar, Meeting Platforms), ensure data compliance, and maintain performance logs. Administrators can activate privacy modes, audit data usage, and manage recovery in case of integration or API failures. They constitute a small technical user group (around 2–3% of users).
Kairo Bot (System Actor)	Kairo Bot is an autonomous virtual participant that automatically joins meetings to record, transcribe, and analyze discussions. It interacts with APIs of platforms like Google Meet or Zoom, captures audio/video streams, generates real-time transcriptions, and produces summaries or action items. It operates without human intervention once configured, enabling seamless automation of meeting intelligence.

Table 1.1: User Classes and Characteristics for Kairo System

1.5 Gantt Chart

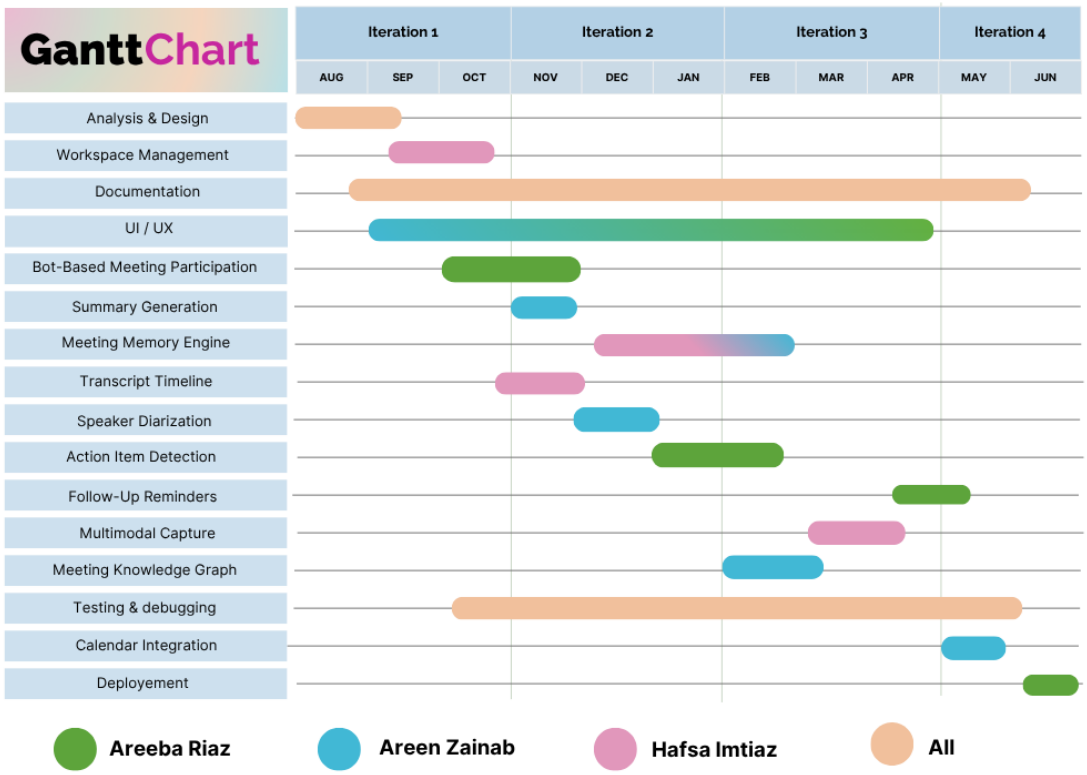

Figure 1.1: Gantt Chart

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-case/Event Response Table/Storyboarding

Since **Kairo** is an interactive end-user application with multiple actors (team members, managers, system components, and external services), the most suitable requirement representation technique is a **Use Case Model**. The following table presents the primary use cases identified during **Iteration #1**. These capture the main interactions between users and the system.

Title	Actor(s)	Preconditions	Postconditions	Main Flow
Manage Workspace & User Roles	Team Lead / Manager	User must be registered and logged in; a workspace must exist or be created.	The workspace is created or updated, and user roles and permissions are stored in the system.	Manager enters workspace details; system validates input and creates workspace. Then, in User Management, the manager adds or removes members, assigns roles, updates permissions, and saves the configuration.
Connect Calendar Service	Team Lead / Manager / Employee	User is a workspace member.	Calendar is linked and synchronized.	User authorizes Google or Outlook integration; upon approval, the system retrieves upcoming meetings.
Join Meeting (Auto / Manual)	Kairo Bot / Team Lead / Manager	A valid meeting link or event must exist.	Bot joins and begins capture.	System auto-joins at the scheduled time or on manual user request.
Real-Time Transcription	Kairo Bot	Bot must be connected to the meeting.	Audio streams sent for transcription.	Bot continuously captures audio and forwards it to the transcription module (WhisperX) for live text output.

Table 2.1: Use Cases for Kairo

Capture Shared Visuals (Slides / Screen)	Kairo Bot	Meeting is in progress with screen sharing.	Visuals are processed and stored.	Bot captures shared visuals, extracts key data (tables, KPIs), and syncs them with the transcript.
Perform Speaker Diarization	System	Transcript must exist.	Text segments are tagged with speaker identity.	System performs voice-based diarization and updates the transcript with labeled segments.
Generate Whisper Mode Micro-Recaps	System	Live transcription is active.	Micro-recaps are displayed.	System generates 5–10 minute summaries and presents them in the meeting UI for context and late joiners.
Generate Post-Meeting Summary	System	Meeting has ended, and a transcript exists.	Summary is generated and stored.	System analyzes transcripts, decisions, and key points to generate layered summaries.
Detect Action Items	System	Processed transcript must exist.	Action items extracted and saved.	System identifies and stores commitments, tasks, and deadlines from the discussion.

Table 2.1: Use Cases for Kairo

Auto-Populate Kanban Board	System	Tasks must be finalized.	Kanban board is populated.	System converts action items into structured Kanban tasks organized by status.
Manage Task	Employee / Manager	Tasks exist on the Kanban board.	Task details updated.	User selects a task, reviews context, updates status or notes, and saves changes; system synchronizes updates and context via the Memory Engine.
Update Meeting Memory & Knowledge Graph	System	Summaries, transcripts, and tasks must be finalized.	Knowledge Graph and meeting memory updated.	System extracts structured data, links related entities, and updates semantic relationships across meetings.
Intelligent Search & Retrieval	Employee / Manager	Meeting and memory data must exist.	Search results retrieved and displayed.	User submits query; system performs contextual, semantic, and speaker-based retrieval to return relevant results.

Table 2.1: Use Cases for Kairo

Activate Privacy & Compliance Mode	Team Lead	Meeting must be in progress.	Sensitive segments excluded from recording.	Team Lead activates Privacy Mode; system pauses recording, logs duration and reason, and resumes upon reactivation.

Table 2.1: Use Cases for Kairo

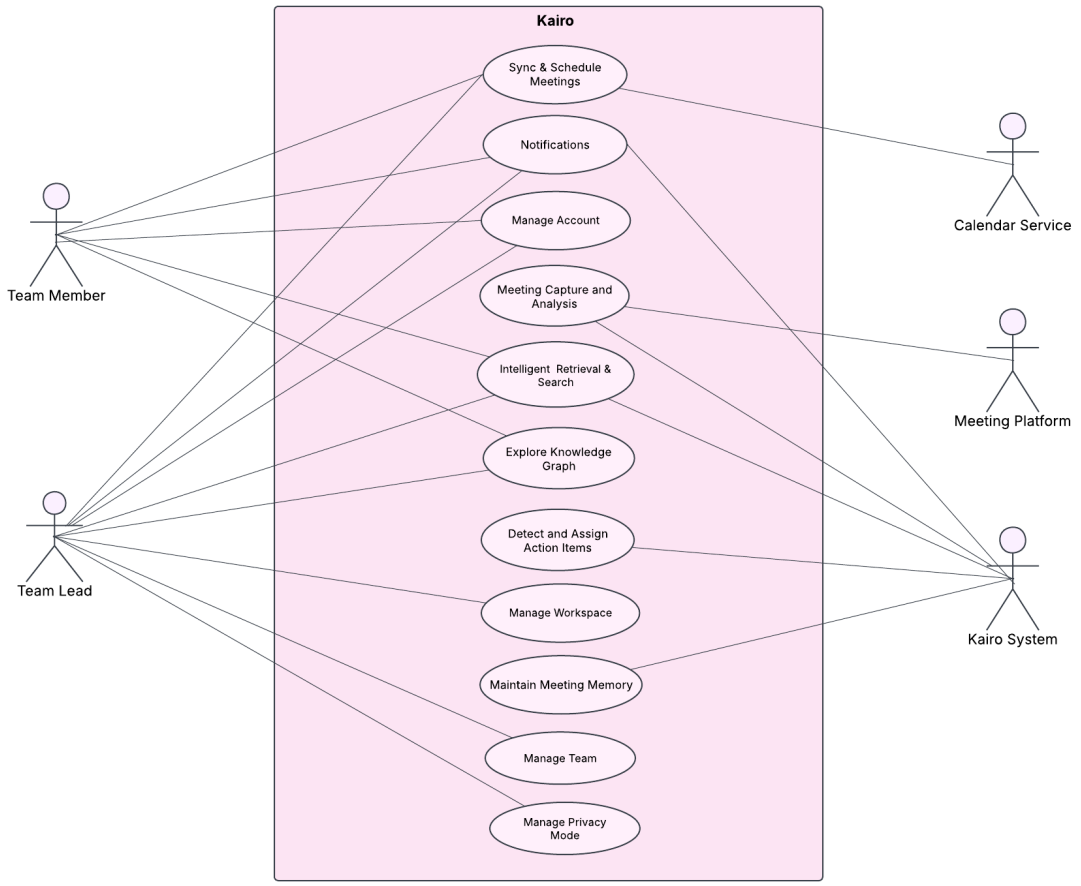

Figure 2.1: Use-Case Diagram

2.2 Functional Requirements

This section describes the functional requirements of the Kairo system, expressed in natural language and organized by system modules. Each module corresponds to a major functional area of the system and includes specific requirements that describe the expected behavior and interactions.

2.2.1 Module 1: Meeting Bot Integration & Workspace Management

This module handles workspace creation, user access, meeting participation, calendar integration, and external tool connections.

1. The system shall allow users to create and join workspaces, and administrators to invite members and assign roles (Admin, Manager, Employee).
2. The system shall enforce secure authentication and role-based access control (RBAC), restricting meeting data visibility and feature access based on user roles.
3. The system shall integrate with Google Calendars to detect upcoming meetings and automatically join supported platforms (e.g., Google Meet, Zoom) as a silent bot participant.
4. The system shall capture audio, video, and screen shares during meetings, and allow manual capture of visuals (e.g., slides, charts, whiteboards).
5. The system shall timestamp and synchronize captured visuals with transcripts and store them securely.
6. The system shall provide pause and resume controls for recording and transcription through Privacy Mode.

2.2.2 Module 2: Transcription and Summarization

This module is responsible for converting speech into text, generating summaries, and providing micro-recaps.

1. The system shall provide real-time transcription of speech into text and store transcripts linked to their corresponding meeting records.
2. The system shall assign speaker identities using diarization and voice embeddings.

3. The system shall generate layered summaries at both executive and detailed levels, including agenda recaps, key discussion points, decisions, and action items.
4. The system shall allow the meeting host to edit, enhance, or finalize the generated summaries.
5. The system shall generate real-time micro-recaps every 5–10 minutes, display them in the chat/UI ticker, and allow late joiners to view past recaps.
6. The system shall provide a collaborative note-taking panel during meetings, where notes are timestamped, attributed to authors, synchronized in real-time, and included in the post-meeting summary.

2.2.3 Module 3: Meeting Memory Engine & Knowledge Graph

This module maintains long-term knowledge continuity, retrieves past meeting context, and visually links related information.

1. The system shall maintain long-term memory of transcripts, summaries, tasks, and decisions.
2. The system shall retrieve contextually relevant past discussions when a new meeting begins.
3. The system shall display unresolved tasks and recurring topics in a Context Panel.
4. The system shall maintain an interactive, visual knowledge graph linking topics, people, tasks, and decisions.
5. The system shall provide an interactive transcript timeline with timestamps.
6. The system shall highlight decisions, tasks, and speaker annotations in transcripts.
7. The system shall allow users to filter and search transcripts by speaker, topic, or task, and jump to specific timestamps.

2.2.4 Module 4: Task Board & Action Item Management

This module extracts tasks from conversations and organizes them into Kanban workflows.

1. The system shall monitor live transcripts to detect task-related commitments, link each task to the corresponding speaker and role, and extract deadlines from natural language expressions.

2. The system shall display identified tasks in a live confirmation panel where users can review, edit, confirm, or discard them.

3. The system shall convert confirmed action items into structured tasks containing a description, assignee, type, priority, and deadline, and display them on a Kanban-style task board.

4. The system shall allow users to manage tasks on the Kanban board, including state transitions (To-Do → In Progress → Done), metadata (assignee, deadline, priority), and real-time drag-and-drop updates.

5. The system shall create a dedicated micro-channel for each task to support ongoing discussions and aggregate related transcripts, notes, and references within that channel.

2.2.5 Module 5: Privacy, Control & Analytics

This module focuses on compliance, advanced search, reminders, and performance analytics.

1. The system shall provide a Privacy Mode that allows users to pause transcription and recording while still logging meeting metadata such as duration and participant information.

2. The system shall restrict Privacy Mode controls to users with the required role permissions.

3. The system shall provide an advanced search capability that allows users to retrieve past meetings by keyword, task, decision, concept, or speaker-specific queries (e.g., "What did Sarah say about deadlines?").

4. The system shall support semantic search using embeddings for contextual understanding.

5. The system shall auto-sync tasks and deadlines extracted from meetings with integrated calendar platforms and send reminders via calendar notifications and in-app alerts, including task details, due dates, and meeting context.

6. The system shall offer an analytics dashboard that summarizes team performance and meeting outcomes through metrics such as task completion rate, meeting duration trends, participation frequency, and unresolved tasks.

7. The system shall allow users to filter analytics data by timeframe, project, or user for more targeted insights.

2.3 Non-Functional Requirements

This section specifies the non-functional requirements for the Kairo system. These quality requirements define the key performance, usability, security, and reliability characteristics that ensure Kairo operates efficiently and consistently. Each requirement is specific, quantitative, and verifiable.

2.3.1 Reliability

These requirements ensure that Kairo consistently performs its intended functions — capturing, transcribing, summarizing, and storing meeting intelligence — with minimal failure.

ID	Requirement	Description & Measurement
1	System Uptime	The system shall have an uptime of $\geq 99.5\%$ per month, excluding scheduled maintenance.
2	Auto-Recovery from Failures	In case of a failure during a meeting (e.g., transcription crash), the system shall auto-recover and resume within 60 seconds without data loss.
3	Data Persistence	To prevent data loss during unexpected outages, all meeting data (transcripts, summaries, tasks) shall be persisted within 5 seconds of generation.
4	Graceful Degradation	If any external integration (e.g., Calendar API) fails, the system shall continue core operations (capture, transcription, summarization) without complete shutdown.
5	Failure Logging & Alerts	Any failure in auto-join, capture, or summarization modules shall trigger real-time error logs and alerts to system admins within 30 seconds.

2.3.2 Usability

These requirements ensure that Kairo is intuitive, learnable, and efficient for diverse user roles (executives, managers, employees).

2.3.3 Performance

These requirements ensure that Kairo operates with real-time responsiveness and scalability, particularly during transcription, summarization, and retrieval tasks.

ID	Requirement	Description & Measurement
1	Ease of Onboarding	New users shall be able to join a workspace and set up an auto-join meeting within 10 minutes without external support.
2	Micro-Recap Accessibility	Late joiners shall access all past micro-recaps within 5 seconds of joining the meeting.
3	Error Recovery	Users shall be able to undo or edit incorrect summaries or extracted tasks within 2 interactions.
4	Accessibility Compliance	The user interface shall provide proper color contrast, keyboard navigation, and screen reader support for accessibility.
5	Multimodal Navigation	Users shall retrieve meeting information via text query, visual graph exploration, or timeline navigation without requiring technical knowledge.

ID	Requirement	Description & Measurement
1	Transcription Latency	Real-time transcription shall display text within 2 seconds of speech.
2	Summarization Speed	Post-meeting summaries shall be generated within 60 seconds for a 60-minute meeting.
3	Micro-Recap Interval	Whisper Mode micro-recaps shall appear within 30 seconds of their scheduled 5–10 minute interval.
4	Search Query Response Time	Semantic search queries (including speaker-based queries) shall return results within 5 seconds.
5	Auto-Join Startup Time	Auto-join shall initiate and connect to a scheduled meeting within 15 seconds of the start time.

2.3.4 Security

These requirements ensure confidentiality, integrity, and controlled access to sensitive meeting data.

2.4 Domain Model

The domain model represents the core structure of the system, identifying its main entities, their attributes, and relationships. It provides a conceptual view of how data and operations interact within the system, serving as a bridge between real-world requirements and the system's design.

ID	Requirement	Description & Measurement
1	Access Control Enforcement	Role-based access control (RBAC) shall ensure 100% prevention of unauthorized access to restricted transcripts, tasks, and graphs.
2	Privacy Mode Compliance	When Privacy Mode is activated, no audio/video/text data shall be captured — only metadata — ensuring zero leakage of sensitive content.
3	Intrusion Detection & Logging	The system shall detect unauthorized access attempts and log them within 10 seconds, generating alerts to administrators.
4	Audit Trail Availability	A complete audit trail (meeting joins, edits, task changes) shall be maintained and retained for at least 1 year.
5	External API Security	All third-party integrations shall use OAuth 2.0 or equivalent secure protocols and require explicit user consent before data sharing.

In this project, the domain model captures various interconnected modules such as meetings, tasks, analytical reports, users, and insights. Each class (or entity) encapsulates attributes and relationships necessary for efficient project and meeting management. For instance:

Users represent individuals interacting with the system, such as employees, managers, or clients.

Meetings store meeting details like date, time, agenda, and participants.

Tasks are linked to meetings and action items, enabling project tracking.

Analytical Reports and Insights provide summaries and evaluations based on meetings and task performance.

Knowledge graphs and transcripts help maintain contextual understanding and decision-making records.

These entities work together to facilitate seamless collaboration, knowledge sharing, and task execution within the organization.

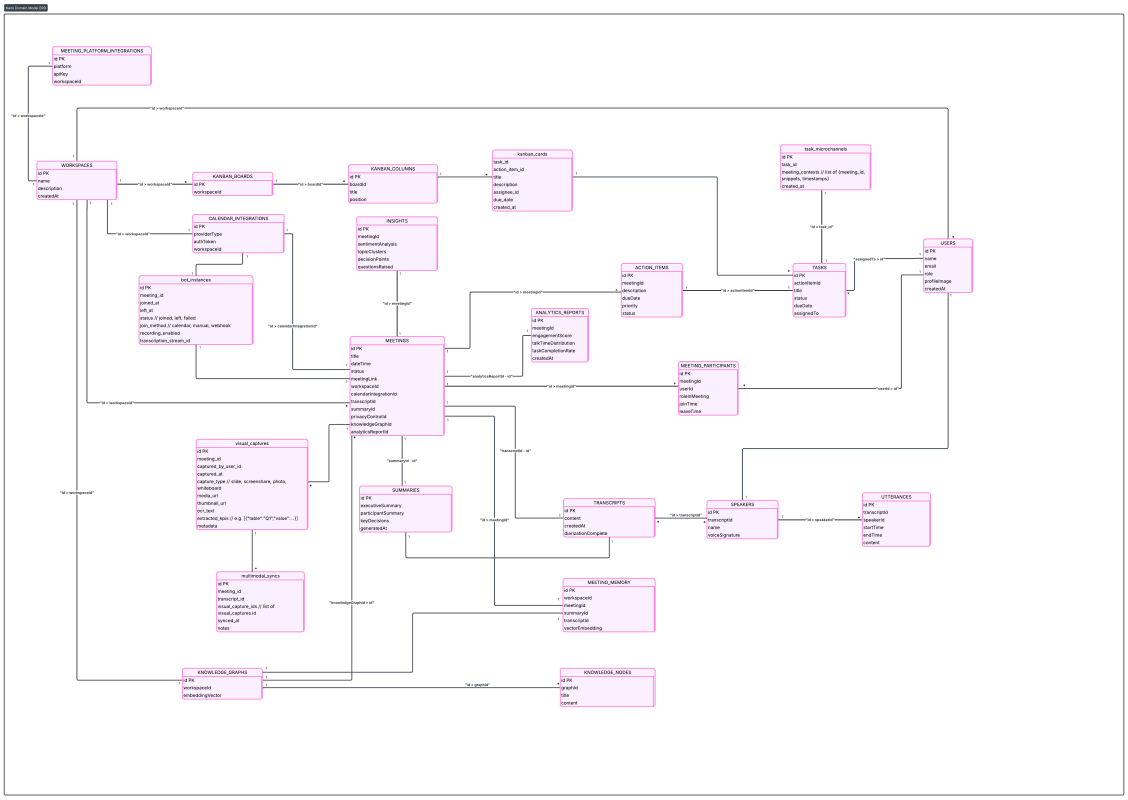

Figure 2.2: Domain Model Diagram of the System

Chapter 3

System Overview

This chapter provides an overview of the overall functionality, structure, and design approach of the system. It explains how different modules interact to deliver the intended features and ensures scalability, maintainability, and a smooth user experience.

3.1 Architectural Design

Kairo follows a robust **three-tier architecture** designed to ensure scalability, modularity, and a clear separation of concerns.

The three primary tiers are the **Presentation Layer**, the **Application Layer**, and the **Data Layer**. This structure enhances scalability and maintainability; and also enables independent evolution of Kairo's intelligence capabilities without disrupting the broader system architecture.

3.1.1 Presentation Layer

The Presentation Layer forms the interface between users and the system. It is where users interact with Kairo through intuitive web-based tools:

- **Web Dashboard:** Provides access to meeting data, insights, and team performance metrics.
- **Kanban Board:** Visualizes and manages action items and tasks automatically generated from meeting outcomes.
- **Search & Graph Explorer:** Enables semantic search and exploration of relationships within the Knowledge Graph.

This layer remains stateless and communicates with backend services through the **API Gateway**, ensuring efficient, secure, and device-agnostic access.

3.1.2 Application Layer

The Application Layer forms the logical and functional backbone of Kairo. It manages all business logic, workflow orchestration, and communication between the other layers and workspace & contextual subsystems, ensuring everything remains synchronized.

Its key responsibilities include:

- **System Coordination:** Orchestrates the end-to-end meeting lifecycle—from scheduling to post-meeting insight delivery.
- **Integration Management:** Connects with external services such as Google Calendar and Zoom to synchronize meetings and automate recording ingestion.
- **User & Workspace Handling:** Manages authentication, authorization, and workspace configurations for teams.
- **Task & Notification Control:** Automates task generation from meeting outcomes and sends relevant summaries or alerts to users.
- **API Gateway:** Acts as a secure entry point for frontend requests and routes them to respective backend microservices.

Kairo Intelligence Core (Logical AI Subsystem) Nested within the Application Layer is the **Kairo Intelligence Core**, which encapsulates all context-driven functionality. It serves as the analytical engine that transforms unstructured meeting data into actionable intelligence. The subsystem includes:

- **Transcription & Summarization:** Converts meeting audio into text and generates concise, structured summaries.
- **Action Item Extraction:** Identifies key decisions and tasks discussed during meetings.
- **Contextual NLP & Semantic Embedding:** Interprets meaning, relationships, and context, enabling intelligent search and retrieval.
- **Knowledge Graph & Insights:** Builds connected representations of people, topics, and outcomes to power analytics and knowledge discovery.

By consolidating these capabilities into a modular subsystem, the Intelligence Core allows Kairo to evolve its contextual abilities independently of other services while maintaining scalability and robust system performance.

3.1.3 Data Layer

The Data Layer is responsible for all persistent storage, indexing, and retrieval operations in Kairo. It comprises multiple specialized databases, each optimized for specific data types. The Workspace DB stores team and user configurations, while the Meeting DB retains transcripts, metadata, and summaries. The Knowledge Graph DB manages relationships and entities extracted by the AI subsystem. The Vector DB (FAISS) stores semantic embeddings for fast similarity search and contextual retrieval. A dedicated Storage module also handles raw meeting recordings, visuals, and supporting documents, ensuring efficient and organized data management across the system.

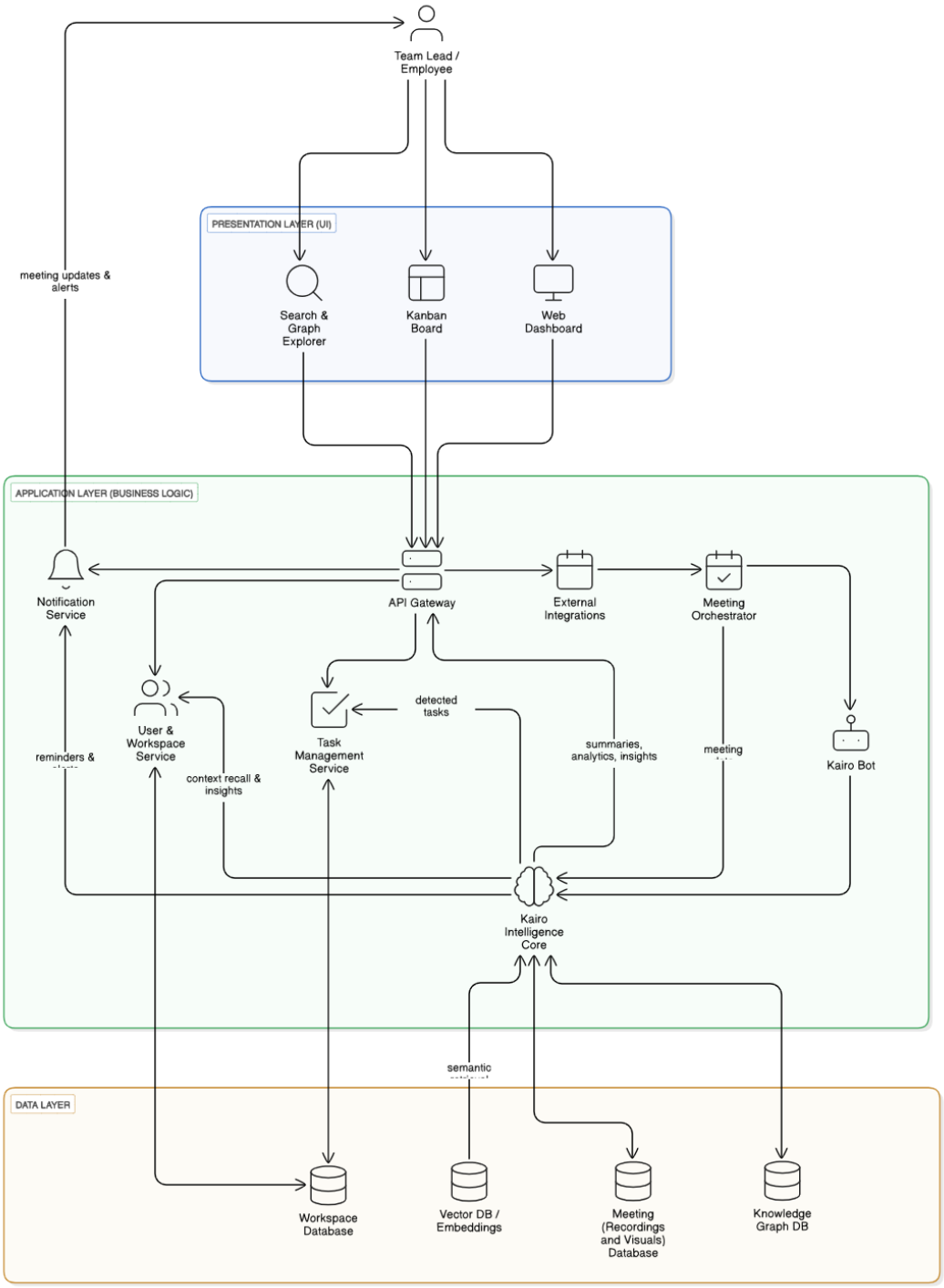

Figure 3.1: System Architectural Design Diagram

3.2 Design Models

This section presents the core design models developed for the Kairo system using an object-oriented development approach.

3.2.1 Class Diagram

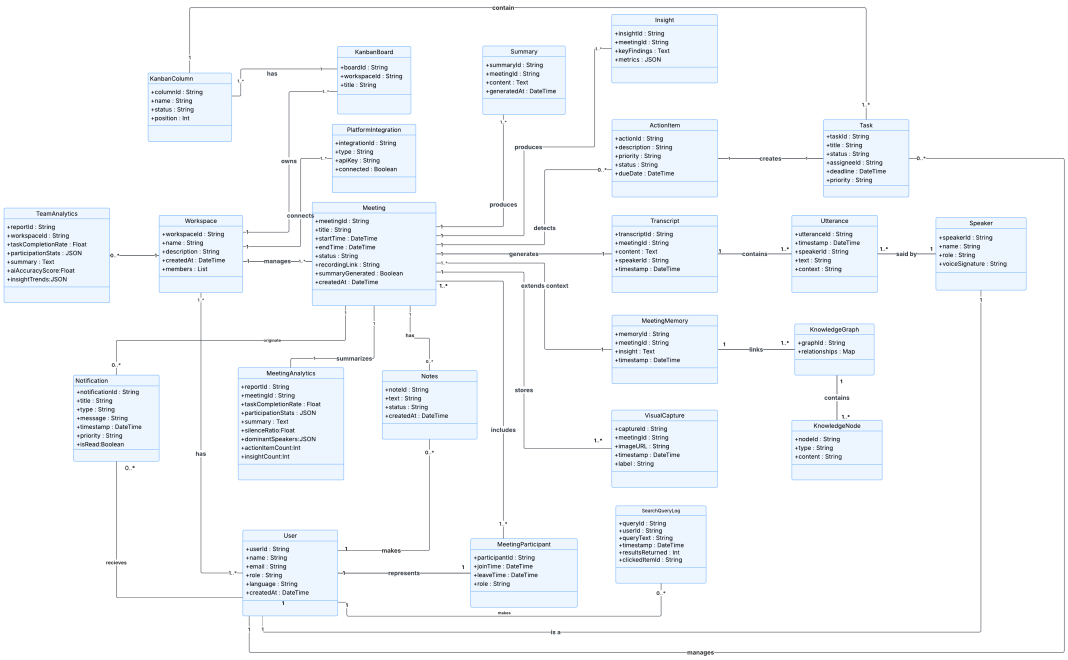

Figure 3.2: Class diagram

3.2.2 Activity Diagram

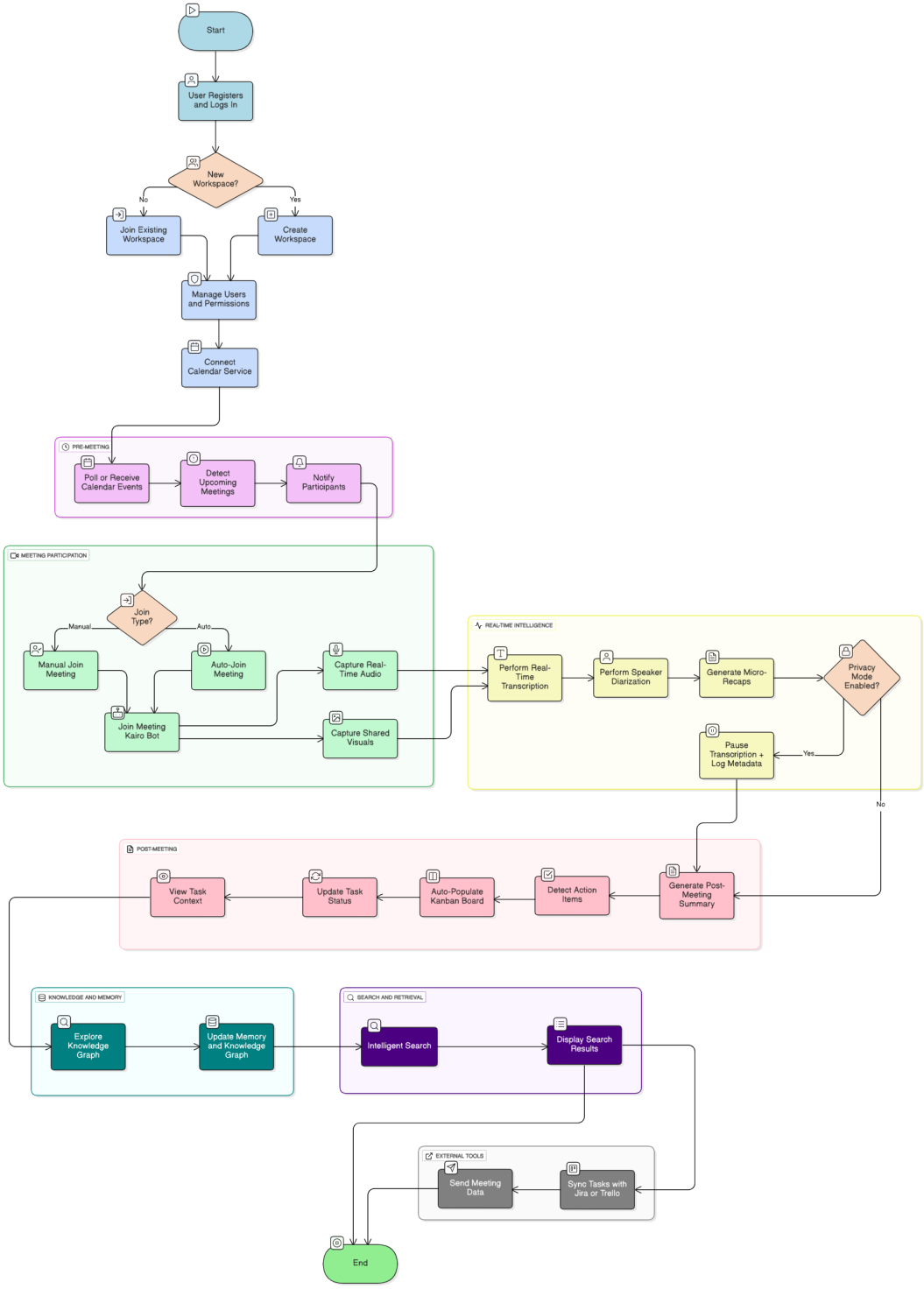

Figure 3.3: Activity Diagram of Kairo showing the meeting lifecycle.

3.2.3 Class-Level Sequence Diagram

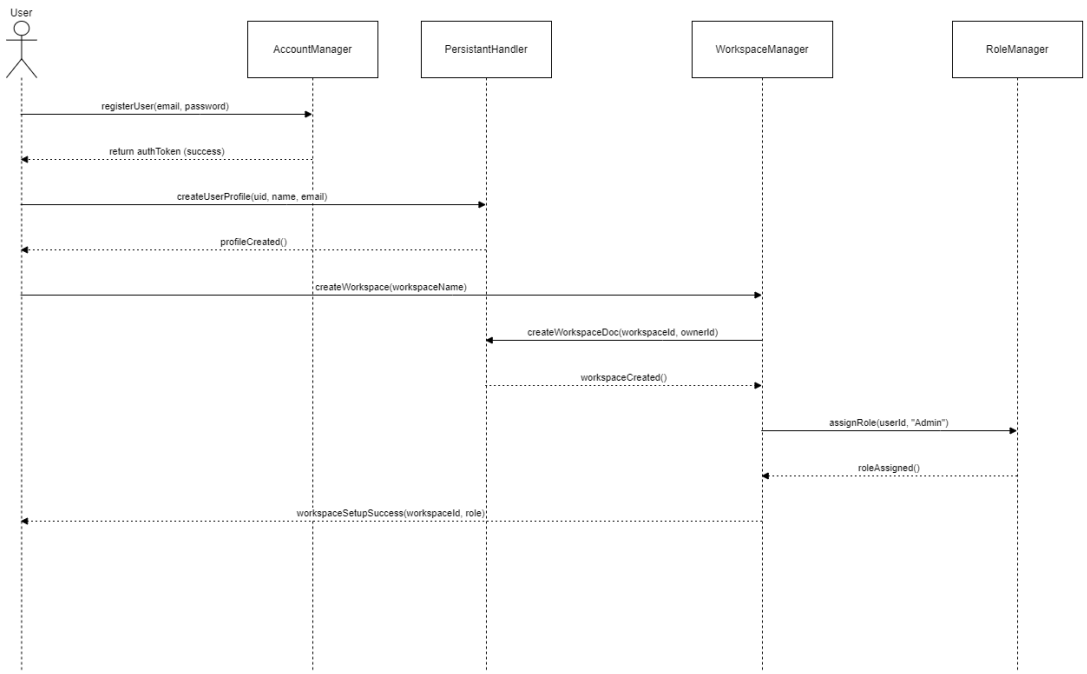

Figure 3.4: Class-level Sequence Diagram for the registration module, highlighting the interactions between user, authentication service, and workspace components.

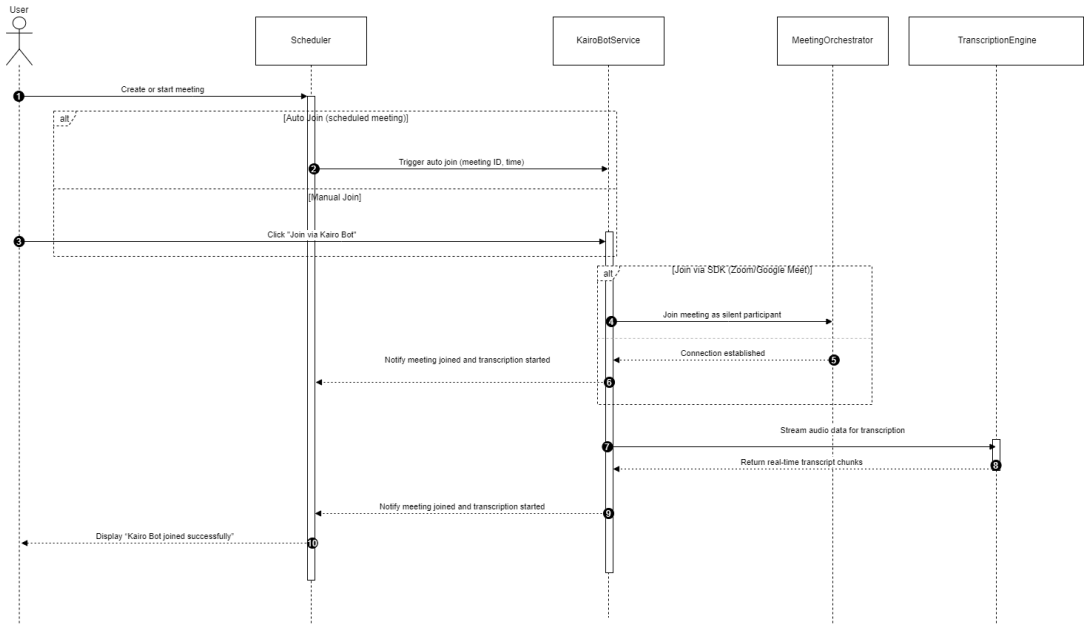

Figure 3.5: Class-level Sequence Diagram illustrating both automatic and manual bot joining processes, followed by transcription initiation.

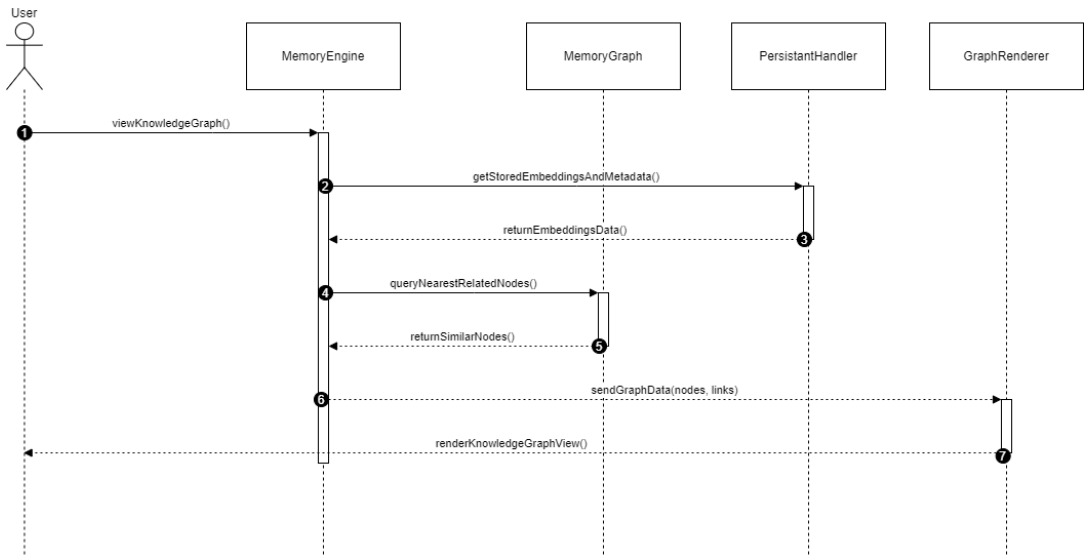

Figure 3.6: Class-level Sequence Diagram for visualizing the Knowledge Graph.

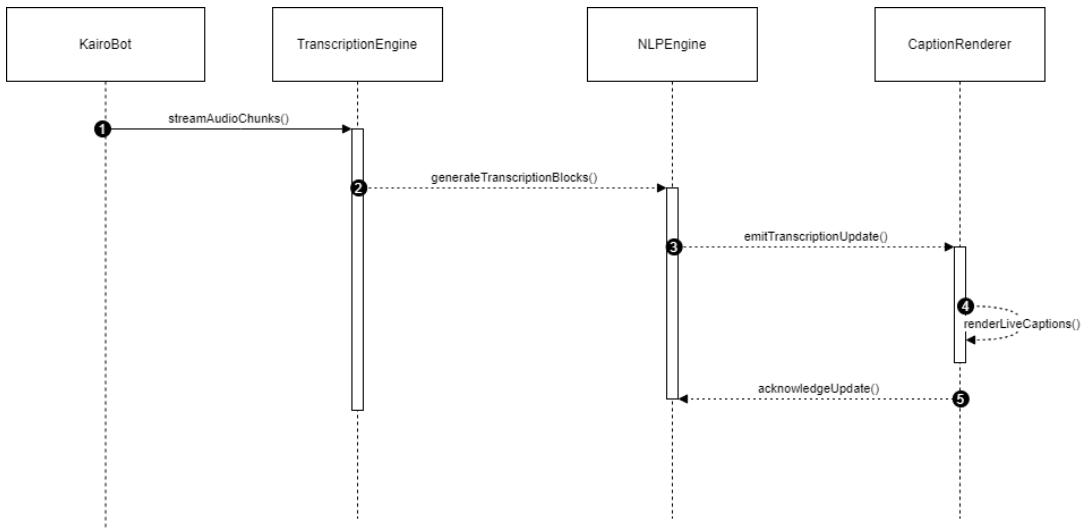

Figure 3.7: Class-Level Sequence Diagram of Live Transcription

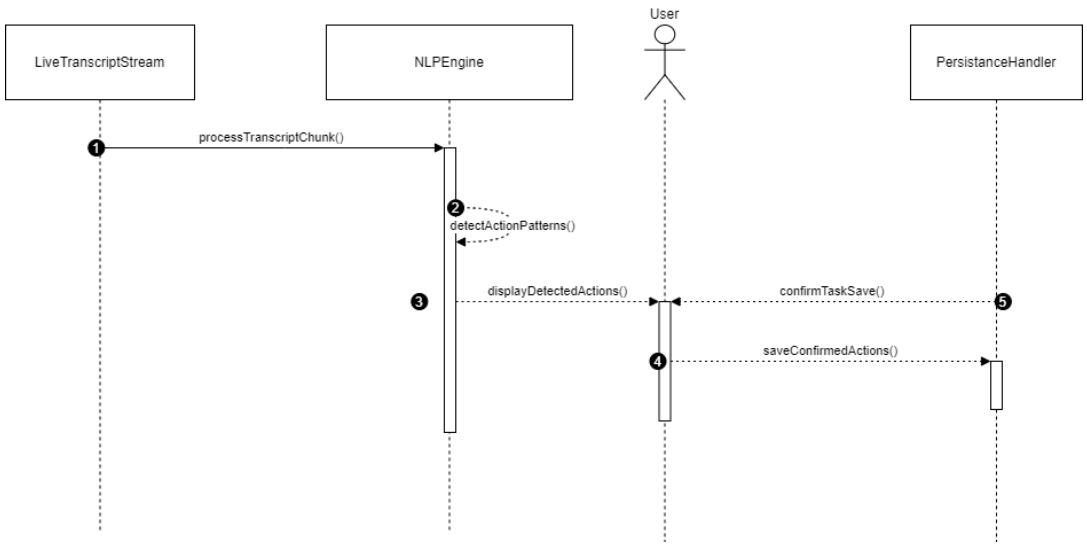

Figure 3.8: Class-level Sequence Diagram illustrating action item detection during meetings.

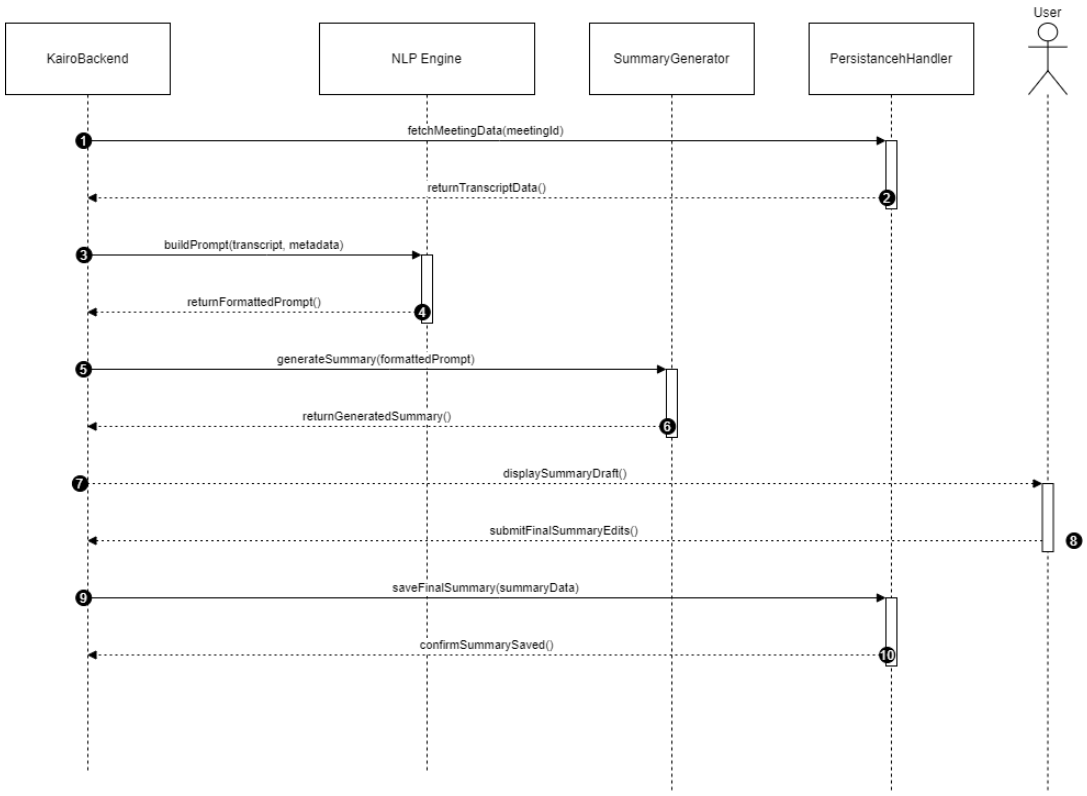

Figure 3.9: Class-level Sequence Diagram for meeting summary generation.

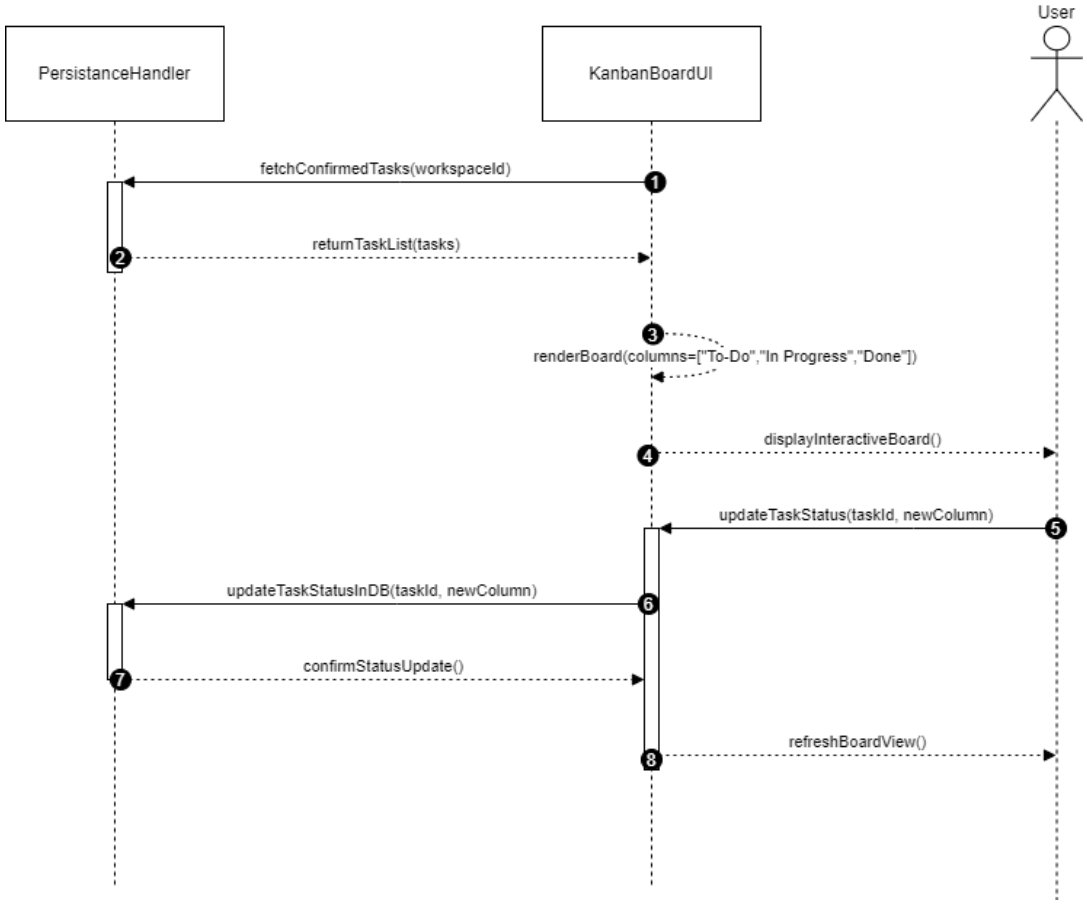

Figure 3.10: Class-level Sequence Diagram depicting Kanban generation within Kairo's task management module.

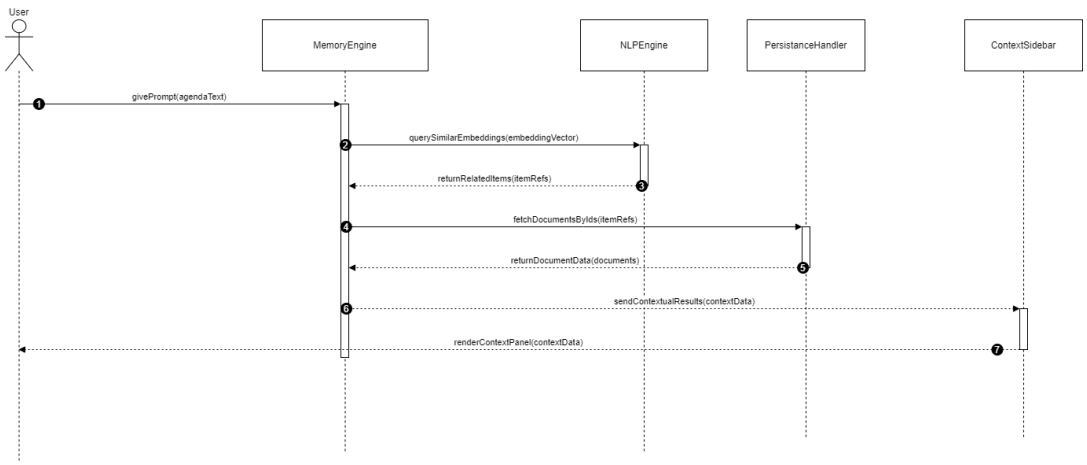

Figure 3.11: Class-level Sequence Diagram depicting the retrieval process from Kairo's Meeting Memory using conversational queries and semantic vector search.

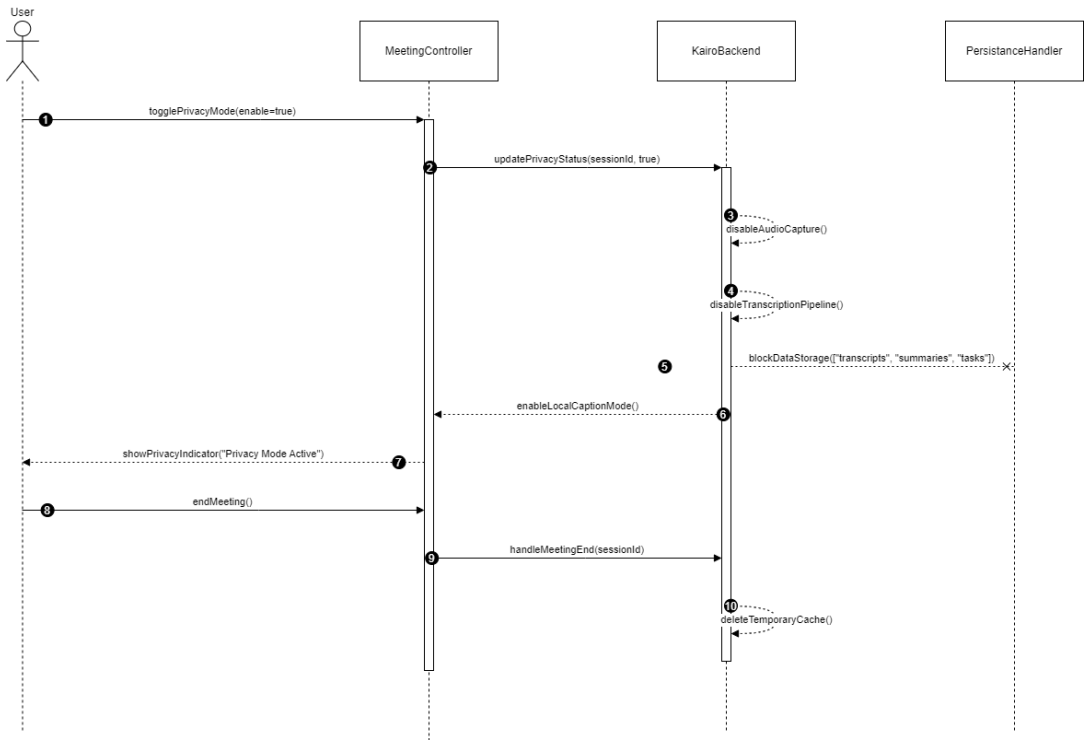

Figure 3.12: Class-level Sequence Diagram illustrating the activation and handling of Privacy Mode in Kairo.

3.2.4 State-Transition Diagram

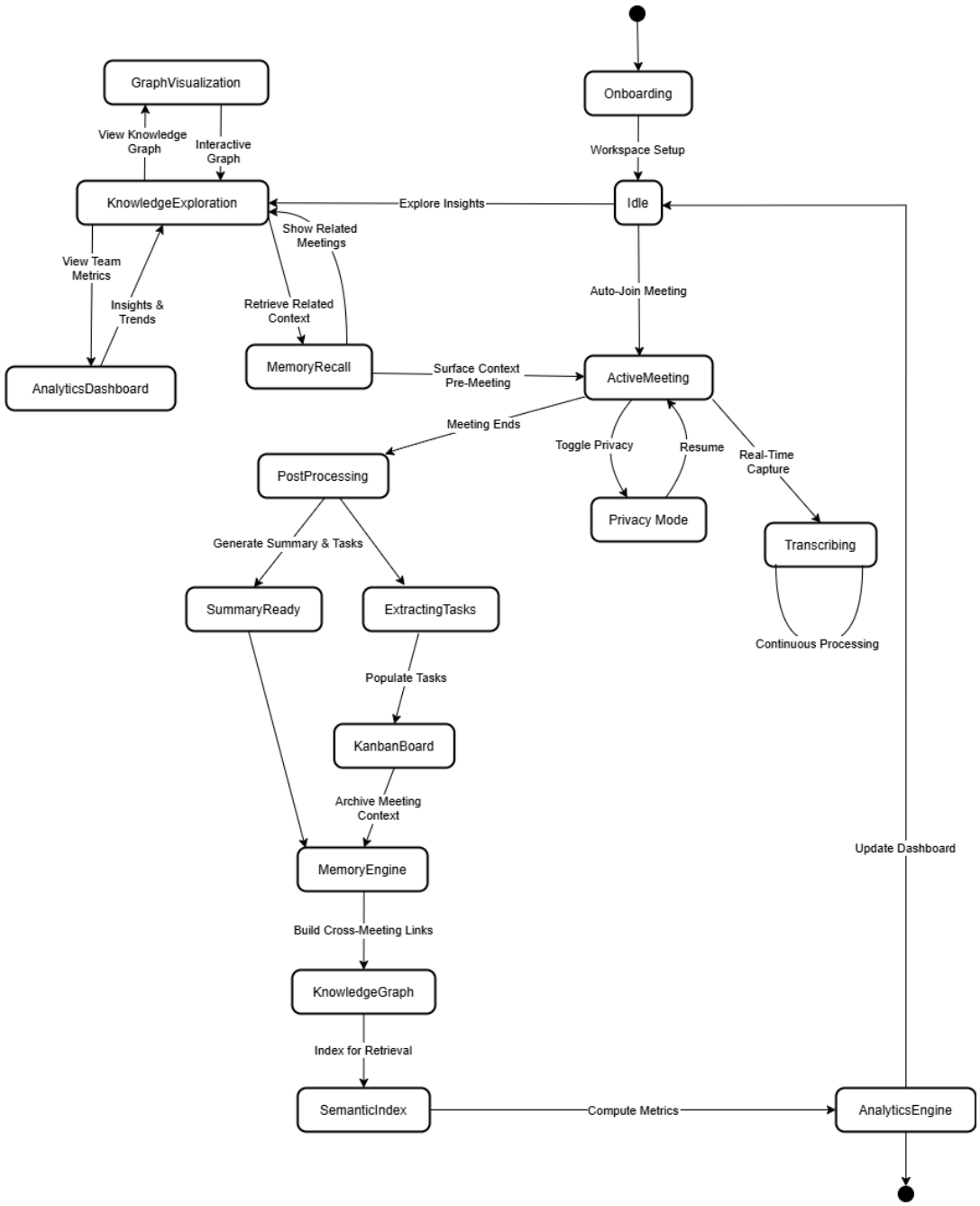

Figure 3.13: State-Transition Diagram of Kairo showing the end-to-end meeting lifecycle.

3.3 Data Design

The data design of **Kairo** defines how meeting-related information is structured, stored, and processed to support its intelligent collaboration features. The system transforms inputs such as meetings, transcripts, action items, and knowledge graphs into structured relational data, ensuring scalability, integrity, and efficient retrieval.

Kairo uses a **PostgreSQL** relational database as its core data store, complemented by a **FAISS** vector database for semantic search and optional cloud storage (e.g., AWS S3) for large audio/video files.

3.3.1 Data Storage Structure

Data is organized into modular layers aligned with system functionality:

- **User & Workspace Layer:** Manages users, roles, permissions, and collaborative workspaces.
- **Meeting Layer:** Stores meeting metadata, participant details, and calendar integration tokens.
- **Processing Layer:** Holds transcripts, summaries, action items, tasks, and privacy settings.
- **Knowledge Layer:** Captures semantic knowledge graphs for cross-meeting insights and search.

All entities are normalized (3NF) to reduce redundancy and enforce relationships.

3.3.2 Major Entities

3.3.3 Data Flow and Processing

The system processes data through the following stages:

1. **Ingestion:** Calendar events and participant details are imported.
2. **Capture:** Kairo Bot records audio/video and captures visuals.
3. **Processing:** Audio is transcribed, and NLP modules generate summaries and tasks.
4. **Storage:** All structured data is saved in relational tables and knowledge graphs.
5. **Retrieval:** Users search, filter, and navigate historical data using semantic queries.

Table 3.1: Key Entities and Their Relationships

Entity	**Purpose**	**Main Relationships**
`users`	Stores user information, roles, and permissions.	Linked to `workspaces`, `meeting_participants`, and `tasks`.
`workspaces`	Defines collaborative environments for teams.	One-to-many with `users`, `meetings`, and `kanban_boards`.
`calendar_integrations`	Stores calendar OAuth tokens.	One-to-many with `meetings`.
`meetings`	Core meeting records with metadata.	Linked to `transcripts`, `summaries`, `participants`, and `action_items`.
`meeting_participants`	Tracks participant attendance and roles.	Many-to-one with `meetings` and `users`.
`transcripts`	Stores meeting transcribed text.	One-to-one with `meetings`.
`summaries`	Contains generated summaries.	One-to-one with `meetings`.
`action_items`	Key tasks identified during meetings.	One-to-many with `tasks`.
`tasks`	Detailed actionable subtasks.	Many-to-one with `action_items`.
`kanban_boards`	Visual task-tracking boards.	One-to-one with `workspaces`.
`knowledge_graphs`	Stores semantic linkages and insights.	One-to-one with `workspaces`.
`privacy_controls`	Controls access to sensitive data.	One-to-one with `meetings`.

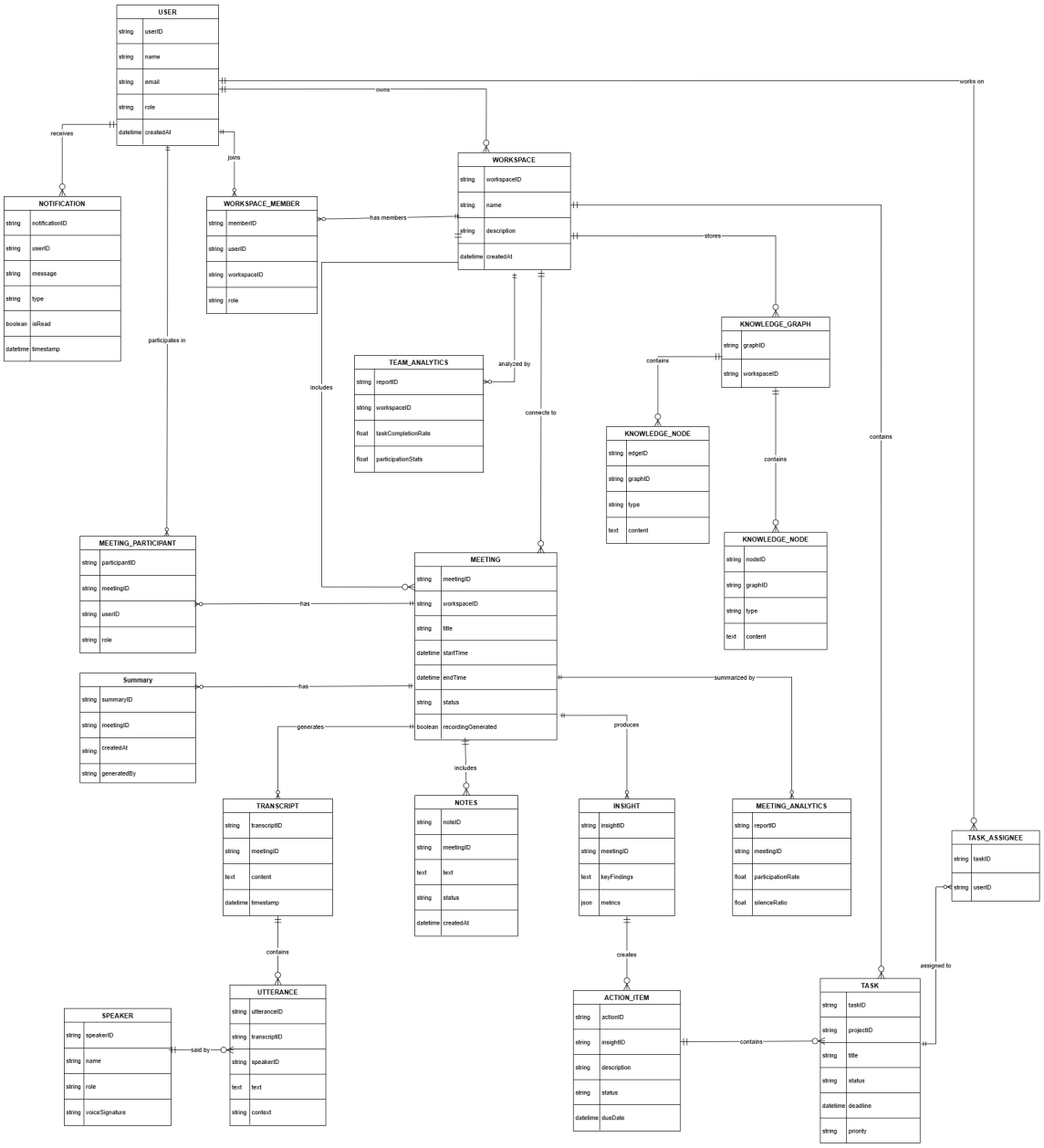

Figure 3.14: Entity-Relationship Diagram

3.3.4 Summary

Kairo's data design ensures that meeting data is captured, structured, and linked across its lifecycle — from scheduling and transcription to action extraction and semantic search. By combining relational models with semantic indexing and scalable storage, the system supports intelligent meeting analytics, contextual retrieval, and enterprise-ready knowledge management.

Bibliography

[1] M. Bain. Whisperx — word-level timestamps and speaker diarization for whisper. GitHub Repository, 2024.

[2] Santosh Bhupathi. Building scalable ai-powered applications with cloud databases: Architectures, best practices and performance considerations. *arXiv preprint arXiv:2504.18793*, April 2025.

[3] L. D. R. Bigtown et al. Summaries, highlights, and action items: Design, implementation, and evaluation of an llm-powered meeting recap system. *arXiv preprint arXiv:2307.15793*, July 2023. Revised.

[4] X. Chen, L. Tankelevitch, R. Vanukuru, A. E. Scott, P. Panda, and S. Rintel. Are we on track? ai-assisted active and passive goal reflection during meetings. *arXiv preprint arXiv:2504.01082*, April 2025.

[5] X. Chen, N. Yap, X. Lu, A. Gunal, and X. Wang. Meetmap: Real-time collaborative dialogue mapping with llms in online meetings. *arXiv preprint arXiv:2502.01564*, February 2025.

[6] Neo4j Graph Database Docs. Knowledge graphs and graph-based retrieval for conversational memory, 2024.

[7] EnterpriseDB (EDB). Postgres: The future-proof data platform for the enterprise. EDB Enterprise Blog, 2024. Discusses Postgres' AI and analytics readiness.

[8] L. Golia and J. Kalita. Action-item-driven summarization of long meeting transcripts. In *Proceedings of the 7th International Conference on NLPIR*, Seoul, December 2023.

[9] JuanMC et al. Diart — real-time diarization framework. GitHub, 2024.

[10] J. Liu, C. Deng, Q. Zhang, Q. Chen, and W. Wang. Meeting action item detection with regularized context modeling. *arXiv preprint arXiv:2303.16763*, March 2023.

[11] Recall.ai. How can i create a zoom bot that joins meetings and interacts as a participant? Recall.ai Blog, 2024.

[12] Resemble.ai. Resemblyzer — voice encoder for speaker embeddings. GitHub Package, 2023.

[13] Y. Weng and J. Wu. Database management systems for artificial intelligence: Comparative analysis of postgresql and mongodb. *World Journal of Advanced Research and Reviews*, 2025. Analysis of PostgreSQL vs MongoDB for AI.